

ATIVIDADE PRÁTICA 1 – SEGURANÇA EM SISTEMAS COMPUTACIONAIS

Tema: Fundamentos de Hashing e Integridade de Dados

INFORMAÇÕES GERAIS

- **Formato de Trabalho:** Grupo de 2 alunos
- **Data de Entrega:** 07/09/2026

OBJETIVOS DO TRABALHO

- Compreender o papel das funções hash na garantia de integridade e autenticação de dados.
- Diferenciar usos seguros e inseguros de hashing em sistemas computacionais.
- Implementar verificadores de integridade e detectores de adulteração de arquivos.
- Analisar limitações práticas de hashing (colisões, rainbow tables, salt).

DIRETRIZES DE ENTREGA

A entrega deve ser realizada em um único arquivo .zip contendo os seguintes elementos:

- **1. Scripts Python:** Código-fonte em Python (.py) devidamente documentado com comentários explicativos. Os scripts devem também ser disponibilizados no Github, com o devido link informado no relatório.
- **2. Relatório Técnico (PDF):** Documento PDF com extensão máxima de 5 páginas contendo os itens abaixo:
 - Respostas fundamentadas às questões teóricas.
 - Explicação detalhada das decisões de implementação.
 - Capturas de tela (prints) da execução bem-sucedida de cada item.
 - Análise crítica e discussão dos resultados obtidos.

CONTEXTO DO CENÁRIO PRÁTICO

Vocês atuam como consultores de segurança da informação e receberam um lote de arquivos de um cliente importante. O cliente suspeita que alguns arquivos foram indevidamente modificados por um invasor e que senhas vazadas da empresa estão sendo utilizadas de forma fraudulenta. Sua tarefa é construir um conjunto de ferramentas e scripts operacionais para investigar a integridade dos dados e testar tecnicamente a robustez dos hashes.

PARTE 1 – CONCEITOS TEÓRICOS

As questões teóricas a seguir devem ser respondidas fundamentadamente no relatório em PDF:

- **1.1 Função Hash Criptográfica:** O que é uma função hash criptográfica? Cite e detalhe três propriedades essenciais.
- **1.2 Hash vs. Cifragem:** Explique a diferença fundamental entre hash e criptografia (cifragem).
- **1.3 Colisões e SHA-1:** O que é uma colisão? Por que o algoritmo SHA-1 é considerado obsoleto na segurança moderna?
- **1.4 Mecanismo de Salt:** O que é um "salt" e por que ele é indispensável no armazenamento seguro de senhas?
- **1.5 Tipos de Ataques a Hashes:** Diferencie detalhadamente ataque de força bruta, ataque de dicionário e ataque com rainbow tables.

PARTE 2 – IMPLEMENTAÇÕES EM PYTHON

RESTRIÇÕES OBRIGATÓRIAS DE IMPLEMENTAÇÃO

- **Bibliotecas de Alto Nível:** Proibido o uso de bibliotecas de alto nível que resolvam a tarefa de forma automatizada (ex: hashlib é permitida APENAS para calcular o hash individual, nunca para verificar integridade completa ou quebrar senhas).
- **Módulos Prontos:** Não utilizar passlib, bcrypt pronto ou ferramentas externas (ex: hashcat, john).
- **Autoria do Código:** Todo o código deve ser desenvolvido inteiramente pela dupla, incluindo a lógica de comparação, geração de salts e construção/pesquisa em dicionários.

Item 2.1 – Verificador de Integridade de Arquivo

Crie um script denominado verificador_integridade.py com as seguintes especificações:

- Receber como argumento via linha de comando o caminho de um diretório (ex: ./dados).
- Calcular o hash SHA-256 de cada arquivo presente no diretório (incluindo subpastas recursivamente).
- Armazenar os hashes em um arquivo hashes.txt no formato exatamente padronizado:

```
caminho_do_arquivo:hash
```

Permitir uma segunda execução passando a flag --verificar. Nesse modo de operação, o script deve:

- Recalcular o hash SHA-256 de todos os arquivos atuais.
- Comparar o estado atual com o registro do arquivo hashes.txt.
- Gerar um relatório de status classificando em: Arquivos novos, Arquivos removidos, Arquivos modificados e Arquivos inalterados.

Exigência Técnica Extra:

- O script deve ser capaz de processar arquivos grandes (> 1 GB) sem estourar a memória RAM, utilizando obrigatoriamente leitura em blocos (chunks).

Item 2.2 – Detector de Senhas Fracas com Hash sem Salt

O cliente forneceu um arquivo `hashes_sem_salt.txt` contendo hashes SHA-256 de senhas (um hash por linha).

Crie um script denominado `quebra_sem_salt.py` com o seguinte fluxo:

- Carregar um dicionário de senhas comuns fornecido separadamente (`senhas_comuns.txt` - uma senha por linha).
- Para cada hash do arquivo do cliente, testar a correspondência contra as senhas do dicionário.
- Exibir o resultado no terminal no formato: `hash:senha_encontrada` ou `hash:NAO_ENCONTRADA`.
- Regra rígida: Implementar manualmente a geração dos hashes (usando `hashlib` apenas para o cálculo unitário) e a lógica de busca/comparação.

Pergunta para o Relatório:

Por que esse método torna-se inviável caso os hashes possuam salt? O que mudaria exatamente na implementação?

Item 2.3 – Implementação de Salt e Validação de Senha

Crie um script `cadastro_verificacao.py` simulando um módulo de cadastro e autenticação de usuários:

- **Modo Cadastro:** Quando executado com `--cadastrar usuario senha`:
 1. Gerar um salt aleatório criptograficamente seguro de 16 bytes (utilizando `os.urandom`).
 2. Concatenar salt + senha (em bytes).
 3. Calcular o hash SHA-256.
 4. Salvar no arquivo `usuarios.txt` no formato: `usuario:salt_hex:hash_hex`
- **Modo Verificação:** Quando executado com `--verificar usuario senha`:
 1. Localizar o registro do usuário no arquivo `usuarios.txt`.
 2. Extrair o salt armazenado, recombina com a senha digitada, calcular o hash e comparar.
 3. Exibir mensagem em tela: "Acesso permitido" ou "Acesso negado".

Exigências de Segurança:

- Nunca armazenar a senha em texto claro (plain text).
- Tratar corretamente as codificações (UTF-8 para conversão de strings e representação Hexadecimal para armazenamento de bytes).

Item 2.4 – Ataque de Dicionário com Salt (Força Bruta Inteligente)

O cliente forneceu um segundo arquivo `hashes_com_salt.txt` no formato: `salt_hex:hash_hex` (um por linha, sem nome de usuário).

Crie um script denominado `quebra_com_salt.py` com as etapas:

- Receber o mesmo dicionário de senhas comuns (`senhas_comuns.txt`).
- Extrair o salt e o hash de cada linha do arquivo fornecido.
- Para cada senha do dicionário, calcular SHA-256(salt + senha).
- Exibir o resultado no terminal no formato: `salt:hash -> senha` ou `salt:hash -> NAO_ENCONTRADA`.

Desafio Técnico (Valendo Bônus):

Implemente uma pré-computação otimizada: caso o mesmo salt apareça reutilizado em múltiplos hashes, calcule o hash de cada senha do dicionário uma única vez para aquele salt específico.

PARTE 3 – ANÁLISE COMPARATIVA

Responda no relatório com base nos experimentos práticos dos itens 2.2 e 2.4:

- Quantas senhas foram quebradas com sucesso em cada um dos cenários?
- Qual o tempo médio de quebra por hash? (Faça estimativas utilizando o comando `time` ou o módulo `time` em Python).
- Explique detalhadamente por que o acréscimo do salt torna o ataque consideravelmente mais lento e menos eficaz.
- Análise de casos: O que aconteceria se dois usuários tivessem a mesma senha e o mesmo salt? E se tivessem a mesma senha mas salts diferentes?
- Proposta de melhoria: Proponha uma evolução conceitual para o sistema de cadastro (item 2.3) utilizando um algoritmo de derivação de chave (ex: PBKDF2) — explicação apenas teórica e conceitual, sem necessidade de implementação.

PARTE 4 – EXTRA (OPCIONAL, COM BÔNUS)

Implemente um script `detector_colisao.py` com a seguinte rotina de testes:

- Utilizar SHA-1 (obsoleto) para gerar hashes de 1.000.000 de strings aleatórias (ex: `os.urandom(10)`).
- Verificar a ocorrência de colisões (instâncias em que strings diferentes produzem o mesmo hash).
- Reportar o número total de colisões encontradas.
- Repetir o exato mesmo experimento substituindo o algoritmo por SHA-256 e comparar.

Pergunta de reflexão: Por que o SHA-256 é estritamente preferível ao SHA-1, mesmo que colisões sejam extremamente raras em um experimento dessa escala?

OBSERVAÇÕES FINAIS

- Não será aceito código-fonte copiado ou gerado integralmente por ferramentas de Inteligência Artificial.
- A qualidade da argumentação técnica no relatório possui o mesmo peso da implementação dos scripts. Expliquem detalhadamente o PORQUÊ de cada escolha de projeto.
- Realize testes graduais: inicie com arquivos e dicionários pequenos e avance para conjuntos maiores (utilize ferramentas como `dd` ou scripts geradores para criar massa de testes).
- Dúvidas conceituais podem ser dirimidas com o professor durante as aulas de apoio.

Bom Trabalho!